

Tema 4: Realización de consultas

Consultas multitable

Contenido

1. Introducción.....	4
2. Producto cartesiano.....	4
3. Consultas multitable	6
3.1. Inner join.....	7
3.2. Natural join	8
3.3. Cross join.....	10
3.4. Outer join.....	11
3.5. Full outer join.....	13
4. Consultas reflexivas	16
5. Otros operadores.....	30
5.1. intersect	30
5.2. except (minus)	31

IES Isidra de Guzmán	
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 4: Realización de consultas	Consultas multitable

5.3. union 32

Figuras

Imagen 1: Inner join	7
Imagen 2: Left join	12
Imagen 3: Right join	12
Imagen 4: Full outer join	14

1. Introducción

En este tema se va a cubrir las consultas a tablas que están relacionadas entre sí.

2. Producto cartesiano

Cuando se especifican varias tablas en la cláusula `where` se obtiene el producto cartesiano de todas las tablas, es decir, todas las combinaciones de filas de cada tabla unida a las filas de otras tablas, por ejemplo:

```
create table roles
(id_rol numeric (2) not null primary key,
rol varchar(50) not null);

create table
sedes
```

IES Isidra de Guzmán		
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026	
Tema 4: Realización de consultas	Consultas multitable	

```

(id_sede numeric (2) not null primary key,
sede varchar(50) not null);

create table
empleados
(id_empleado numeric (2) not null primary key, nombre varchar(50) not null,
id_rol numeric (2) not null, id_sede numeric (2) null, constraint
fk_empleados_rols foreign key (id_rol) references roles(id_rol),
constraint fk_empleados_sedes foreign key (id_sede) references
sedes(id_sede)
);

insert into roles (id_rol, rol)
values
(1, 'técnico'),
(2, 'supervisor'),
(3, 'director');

insert into sedes (id_sede, sede)
values
(10, 'Madrid'),
(11, 'Málaga'),
(12, 'Segovia');

insert into empleados (id_empleado, nombre, id_rol, id_sede)
values
(20, 'Pedro', 1, 10),
(21, 'Verónica', 2, 11), (22,
'Eva', 2, null);

```

Si hacemos una select de las 3 tablas obtendremos toda las combinaciones de filas de todas las tablas:

IES Isidra de Guzmán	
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 4: Realización de consultas	Consultas multitable

```
select * from roles, sedes,
empleados;
```

id_rol	rol	id_sede	sede	id_empleado	nombre	id_rol	id_sede
1	técnico	12	Segovia	20	Pedro	1	10
2	supervisor	12	Segovia	20	Pedro	1	10
3	director	12	Segovia	20	Pedro	1	10
1	técnico	11	Málaga	20	Pedro	1	10
2	supervisor	11	Málaga	20	Pedro	1	10
3	director	11	Málaga	20	Pedro	1	10
1	técnico	10	Madrid	20	Pedro	1	10
2	supervisor	10	Madrid	20	Pedro	1	10
3	director	10	Madrid	20	Pedro	1	10
1	técnico	12	Segovia	21	Verónica	2	11
2	supervisor	12	Segovia	21	Verónica	2	11
3	director	12	Segovia	21	Verónica	2	11
1	técnico	11	Málaga	21	Verónica	2	11
2	supervisor	11	Málaga	21	Verónica	2	11
3	director	11	Málaga	21	Verónica	2	11
1	técnico	10	Madrid	21	Verónica	2	11
2	supervisor	10	Madrid	21	Verónica	2	11
3	director	10	Madrid	21	Verónica	2	11
1	técnico	12	Segovia	22	Eva	2	NULL
2	supervisor	12	Segovia	22	Eva	2	NULL
3	director	12	Segovia	22	Eva	2	NULL
1	técnico	11	Málaga	22	Eva	2	NULL
2	supervisor	11	Málaga	22	Eva	2	NULL
3	director	11	Málaga	22	Eva	2	NULL
1	técnico	10	Madrid	22	Eva	2	NULL
2	supervisor	10	Madrid	22	Eva	2	NULL
3	director	10	Madrid	22	Eva	2	NULL

IES Isidra de Guzmán	
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 4: Realización de consultas	Consultas multitable

3. Consultas multitable

Realizar una consulta a varias tablas sin tener en cuenta su relación entre sí no tiene ninguna utilidad. Por ello a la hora de consultar información de varias tablas es necesario utilizar los campos que las relacionan (`join`). Existen cinco tipos de `joins`:

3.1. Inner join

Esta consulta parte del producto cartesiano de las tablas involucradas y posteriormente solo se muestran las filas que cumplen las condiciones que se especifiquen en el `join`.

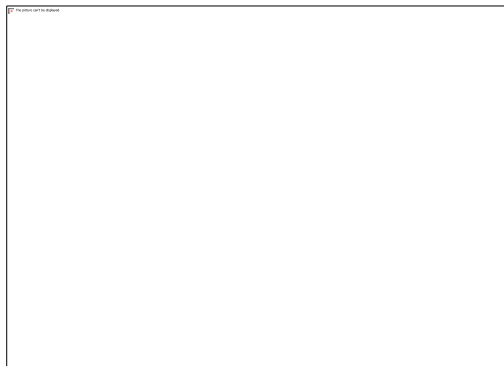


Imagen 1: Inner join

La sintaxis es:

```
select ... from tabla_A INNER JOIN
tabla_B ON
tabla_A.columna=tabla_B.columna;
```

Por ejemplo, para mostrar cada empleado y su sede utilizaríamos la columna `id_sede` para relacionar ambas tablas:

IES Isidra de Guzmán		
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026	
Tema 4: Realización de consultas	Consultas multitable	

```

select e.nombre, s.sede
from empleados e inner
join sedes s on
e.id_sede=s.id_sede; +-
-----+-----+ |
nombre    | sede      |
+-----+-----+
| Pedro    | Madrid    |
| Verónica | Málaga    | +-----+-----+

```

Si deseamos mostrar cada empleado con su sede y su rol deberemos consultar involucras las tres tablas en el join:

```

select e.nombre, s.sede, r.rol
from empleados e inner join
sedes s on e.id_sede=s.id_sede
inner join roles r on
e.id_rol=r.id_rol;

+-----+-----+-----+
| nombre    | sede      | rol          |
+-----+-----+-----+
| Pedro     | Madrid    | técnico      |
| Verónica  | Málaga    | supervisor   | +-----+-----+

```

3.2. Natural join

IES Isidra de Guzmán		
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026	
Tema 4: Realización de consultas	Consultas multitable	

Es igual que `inner join` excepto que no se especifica la condición sino que el SGBD utiliza columnas con el mismo nombre para realizar el `join` (por lo tanto si el nombre de las columnas que relacionan las tablas son distintas no funcionará). Su sintaxis es:

```
select ... from tabla_A NATURAL JOIN
tabla_B;
```

Por ejemplo:

```
select e.nombre, s.sede
from empleados e
natural join sedes s;
```

nombre	sede
Pedro	Madrid
Verónica	Málaga

```
select e.nombre, s.sede,
r.rol from empleados e natural
join sedes s natural join
roles r
```

nombre	sede	rol
Pedro	Madrid	técnico
Verónica	Málaga	supervisor

IES Isidra de Guzmán		
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026	
Tema 4: Realización de consultas	Consultas multitable	

3.3. Cross join

El resultado es el producto cartesiano sin ningún filtro, su sintaxis es:

```
select ... from tabla_A CROSS JOIN
```

```
tabla_B;
```

```
select e.nombre, s.sede,
r.rol from empleados e cross
join sedes s cross join roles
r;
```

Por ejemplo:

```
+-----+-----+-----+
| nombre | sede  | rol    |
+-----+-----+-----+
| Pedro  | Segovia | técnico |
| Verónica | Segovia | técnico |
| Eva    | Segovia | técnico |
| Pedro  | Málaga  | técnico |
| Verónica | Málaga  | técnico |
| Eva    | Málaga  | técnico |
| Pedro  | Madrid  | técnico |
| Verónica | Madrid  | técnico |
| Eva    | Madrid  | técnico |
| Pedro  | Segovia | supervisor |
| Verónica | Segovia | supervisor |
| Eva    | Segovia | supervisor |
| Pedro  | Málaga  | supervisor |
| Verónica | Málaga  | supervisor |
```

IES Isidra de Guzmán	
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 4: Realización de consultas	Consultas multitable

Eva	Málaga	supervisor	
Pedro	Madrid	supervisor	
Verónica	Madrid	supervisor	
Eva	Madrid	supervisor	
Pedro	Segovia	director	
Verónica	Segovia	director	
Eva	Segovia	director	
Pedro	Málaga	director	
Verónica	Málaga	director	
Eva	Málaga	director	
Pedro	Madrid	director	
Verónica	Madrid	director	
Eva	Madrid	director	

+-----+-----+-----+

3.4. Outer join

Sirven para mostrar los registros del `inner join` más los registros de una de las tablas que no estén relacionados. Dependiendo de la tabla de la que se deseen obtener los registros que no están relacionados se especifica `LEFT` o `RIGHT`.

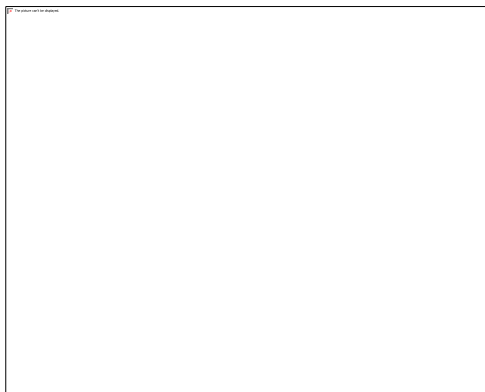


Imagen 2: Left join

IES Isidra de Guzmán	
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 4: Realización de consultas	Consultas multitable

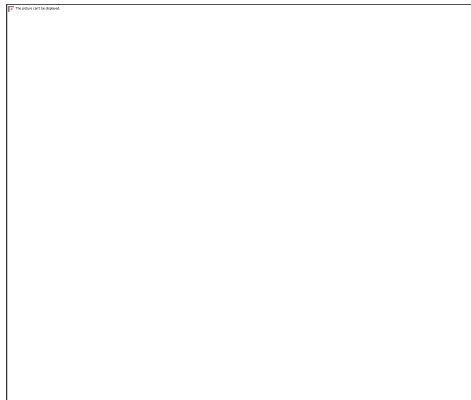


Imagen 3: Right join

Su sintaxis es:

```
select ... from tabla_A LEFT|RIGHT OUTER
JOIN tabla_B
ON tabla_A.columna=tabla_B.columna;
```

Por ejemplo, se desean obtener todos los nombres de las sedes con sus empleados (según la estructura de las tablas podría haber sedes sin empleados):

```
select s.sede, e.nombre from
sedes s left outer join
empleados e on
s.id_sede=e.id_sede;
```

```
+-----+-----+
| sede   | nombre   |
+-----+-----+
| Madrid | Pedro    |
| Málaga | Verónica |
| Segovia | NULL     | +-----+-----+
```

IES Isidra de Guzmán	
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 4: Realización de consultas	Consultas multitable

Obtener todos los nombres de los roles con sus empleados (según la estructura de las tablas podría haber roles sin empleados):

```
select r.rol, e.nombre
from empleados e right
outer join roles r on
e.id_rol=r.id_rol;
```

```
+-----+-----+
| rol      | nombre    |
+-----+-----+
| técnico  | Pedro     |
| supervisor | Verónica  |
| director | NULL      | +-----+-----+
```

3.5. Full outer join

Es una combinación de inner, left y right join.

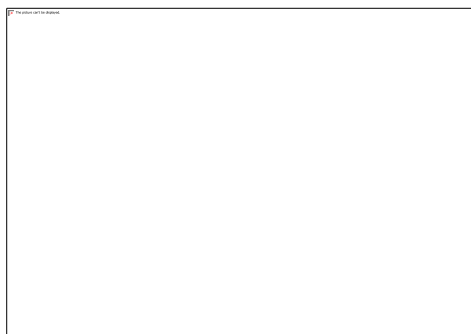


Imagen 4: Full outer join

Su sintaxis es:

IES Isidra de Guzmán	
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 4: Realización de consultas	Consultas multitable

```
select ... from tabla_A FULL OUTER
JOIN tabla_B ON
tabla_A.columna=tabla_B.columna;
```

En MySQL no existe este join porque se puede utilizar el operado UNION junto con LEFT y RIGHT OUTER JOIN:

El operador UNION une 2 consultas:

```
select ... from
tabla_A
UNION [ALL]
select ... from
tabla_B
```

Si se especifica ALL no se eliminan filas repetidas;

Por lo tanto para realizar un full outer join se puede hacer la consulta de la siguiente manera:

```
select ... from tabla_A LEFT OUTER JOIN
tabla_B ON
tabla_A.columna=tabla_B.columna UNION
select ... from tabla_A RIGHT OUTER
JOIN tabla_B ON
tabla_A.columna=tabla_B.columna
```

Por ejemplo, deseamos obtener roles, sedes y empleados teniendo en cuenta que puede haber roles y sedes sin empleados y empleados que no tengan sede

IES Isidra de Guzmán	
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 4: Realización de consultas	Consultas multitable

```
select s.sede, e.nombre from
sedes s left outer join
empleados e on
s.id_sede=e.id_sede union
select s.sede, e.nombre from
sedes s right outer join
empleados e on
s.id_sede=e.id_sede;
```

```
+-----+-----+
| sede   | nombre   |
+-----+-----+
| Madrid | Pedro    |
| Málaga | Verónica |
| Segovia | NULL     |
| NULL   | Eva      |
+-----+-----+
```

4. Consultas reflexivas

Son consultas donde se hace `join` con la misma tabla, por ejemplo:

```
create table empleados ( dni char(9) not null, nombre
varchar(50) not null, dni_superior char(9) null,
constraint pk_empleados primary key (dni), constraint
fk_empleado_superior foreign key (dni_superior)
references empleados (dni)
```

IES Isidra de Guzmán	
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 4: Realización de consultas	Consultas multitable

);

```
#Test insert into
empleados ( dni,
nombre, dni_superior)
values
('01000000J', 'Jefe Supremo ', null),
('02000000I', 'Jefe Intermedio', '01000000J'),
('03000000E', 'Empleado Uno', '02000000I'),  ('04000004E',
'Empleado Dos', '02000000I');
select empleado.nombre as
nombre, superior.nombre as
nombre_superior from empleados
empleado left join empleados
superior on
empleado.dni_superior=superior.dn
i; +-----+-----+
-----+
| nombre          | nombre_superior |
+-----+-----+
| Jefe Supremo    | NULL            |
| Jefe Intermedio | Jefe Supremo    |
| Empleado Uno    | Jefe Intermedio |
| Empleado Dos    | Jefe Intermedio | +-----+-----+
```

IES Isidra de Guzmán		
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026	
Tema 4: Realización de consultas	Consultas multitable	

Actividad 1

Dada la siguiente tabla y datos:

```
create table clientes (
id_cliente int primary key,
nombre varchar(50),      ciudad
varchar(50)
); create table mascotas (
id_mascota int primary key,
nombre_mascota varchar(50),
especie varchar(20),
id_cliente int
); create table citas (
id_cita int primary key,
id_mascota int,      fecha
date,      coste
decimal(10,2)
); insert into clientes
values (1, 'Carlos Ruiz',
'Madrid'),
(2, 'Laura Gil', 'Barcelona'),
(3, 'Pedro Sanz', 'Madrid'), (4,
'Marta Soto', 'Sevilla');
insert into mascotas
values (10, 'Rex',
'Perro', 1),
(20, 'Michi', 'Gato', 1),
(30, 'Piolín', 'Pájaro', 2),
(40, 'Firulais', 'Perro', 3),
```

IES Isidra de Guzmán	
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 4: Realización de consultas	Consultas multitable

```
(50, 'Nemo', 'Pez', NULL); # Mascota sin dueño (en adopción)

insert into citas values

(100, 10, '2024-01-10', 50.00),

(101, 10, '2024-02-15', 30.00),

(102, 20, '2024-01-20', 45.00),

(103, 30, '2024-03-01', 25.00),

(104, 60, '2024-03-10', 40.00); # Cita de mascota que ya no existe
```

Realizar las siguientes sentencias:

- Nombre del cliente, el nombre de su mascota y la fecha de sus citas (solo deben aparecer si tienen los tres datos).
- `SELECT cl.nombre AS 'Cliente', m.nombre_mascota AS 'Mascota', c.fecha AS 'Fecha Cita' FROM clientes cl INNER JOIN mascotas m ON cl.id_cliente = m.id_cliente INNER JOIN citas c ON m.id_mascota = c.id_mascota;`
- Muestra todos los clientes y el nombre de sus mascotas. Si un cliente no tiene mascota debe aparecer su nombre y "<sin mascota>".
- `SELECT cl.nombre AS 'Cliente', IFNULL(m.nombre_mascota, '<sin mascota>') AS 'Mascota' FROM clientes cl LEFT JOIN mascotas m ON cl.id_cliente = m.id_cliente;`
- Muestra todas las mascotas y el nombre de su dueño. Si una mascota no tiene dueño deberá aparecer "<sin dueño>".
- `SELECT m.nombre_mascota AS 'Mascota', IFNULL(cl.nombre, '<sin dueño>') AS 'Dueño' FROM mascotas m LEFT JOIN clientes cl ON m.id_cliente = cl.id_cliente;`
- Muestra el nombre de cada cliente y el coste total que ha gastado en citas.
- `SELECT cl.nombre AS 'Cliente', SUM(c.coste) AS 'Coste Total' FROM clientes cl INNER JOIN mascotas m ON cl.id_cliente = m.id_cliente INNER JOIN citas c ON m.id_mascota = c.id_mascota GROUP BY cl.nombre;`
- Lista todas las mascotas que tengan dueño, la fecha de sus citas y el nombre de su dueño, ordenado por fecha de cita. Deben aparecer incluso las mascotas que nunca han tenido una cita.
- `SELECT m.nombre_mascota AS 'Mascota', c.fecha AS 'Fecha Cita', cl.nombre AS 'Dueño' FROM mascotas m INNER JOIN clientes cl ON m.id_cliente = cl.id_cliente LEFT JOIN citas c ON m.id_mascota = c.id_mascota ORDER BY c.fecha ASC;`

IES Isidra de Guzmán		
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026	
Tema 4: Realización de consultas	Consultas multitable	

Actividad 2

Dada la siguiente tabla y datos:

```
create table autores (
id_autor int primary key,
nombre_autor varchar(50),
nacionalidad varchar(30)
); create table libros (
id_libro int primary key,
titulo varchar(100),
id_autor int
); create table
prestamos (
id_prestamo int primary
key,      id_libro int,
nombre_usuario
varchar(50),
fecha_prestamo date
); insert into autores
values
(1, 'Miguel de Cervantes', 'Española'),
(2, 'Gabriel García Márquez', 'Colombiana'),
(3, 'Isabel Allende', 'Chilena');
insert into libros values (10,
'Don Quijote de la Mancha', 1),
(20, 'Cien años de soledad', 2),
(30, 'El amor en los tiempos del cólera', 2),
(40, 'La casa de los espíritus', 3),
(50, 'Libro Huérfano', NULL); # Libro sin autor registrado
```

IES Isidra de Guzmán		
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026	
Tema 4: Realización de consultas	Consultas multitable	

```

insert into prestamos values (100,
10, 'Juan Pérez', '2024-01-05'),
(101, 10, 'Ana López', '2024-02-10'), # El mismo libro prestado otra vez
(102, 20, 'Juan Pérez', '2024-01-15'), # El mismo usuario lleva otro libro
(103, 30, 'Luis Gómez', '2024-03-01'),
(104, 99, 'Usuario Fantasma', '2024-03-05'); # Préstamo de libro inexistente

```

Realizar las siguientes sentencias:

- Muestra el nombre del autor, el título del libro y el nombre del usuario que lo pidió. Solo registros con toda la información.
- `SELECT a.nombre_autor, l.titulo, p.nombre_usuario FROM prestamos p INNER JOIN libros l ON p.id_libro = l.id_libro INNER JOIN autores a ON l.id_autor = a.id_autor;`

IES Isidra de Guzmán	
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 4: Realización de consultas	Consultas multitable

-

-

Obtén la lista de usuarios (sin repetición) que han tomado prestados libros de autores de nacionalidad colombiana.

```
SELECT DISTINCT p.nombre_usuario FROM prestamos p INNER JOIN libros l ON
p.id_libro = l.id_libro INNER JOIN autores a ON l.id_autor = a.id_autor WHERE
a.nacionalidad = 'Colombiana';
```

Lista todos los autores y los títulos de sus libros, si un libro no tiene autor se debe mostrar "ANÓNIMO".

```
SELECT IFNULL(a.nombre_autor, 'ANÓNIMO') AS 'Autor', l.titulo FROM libros l LEFT JOIN
autores a ON l.id_autor = a.id_autor;
```

- Cuenta cuántos autores distintos tienen libros que han sido prestados al menos una vez.
- ```
SELECT COUNT(DISTINCT a.id_autor) AS 'Total Autores Prestados' FROM autores a INNER JOIN
libros l ON a.id_autor = l.id_autor INNER JOIN prestamos p ON l.id_libro = p.id_libro;
```
- • Muestra el nombre de los usuarios (sin repetición) que han leído algún libro de 'Gabriel García Márquez'.
- ```
SELECT DISTINCT p.nombre_usuario
FROM prestamos p
INNER JOIN libros l ON p.id_libro = l.id_libro
INNER JOIN autores a ON l.id_autor = a.id_autor
WHERE a.nombre_autor = 'Gabriel García Márquez';
```

Actividad 3

Dada la siguiente tabla y datos:

```
create table directores (
id_director int primary key,
nombre_director varchar(50),
pais_origen varchar(30)
) ; create table peliculas (
id_pelicula int primary key,
```

IES Isidra de Guzmán	
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 4: Realización de consultas	Consultas multitable

```

titulo varchar(100),      genero
varchar(30),      id_director int
) ; create table salas (      id_sala int primary key,
nombre_sala varchar(20),      id_pelicula int, # película que se
está proyectando actualmente      capacidad int

) ; insert into directores values  (1,
'Christopher Nolan', 'Reino Unido'),
(2, 'Steven Spielberg', 'EEUU'),
(3, 'Greta Gerwig', 'EEUU'), (4,
'Pedro Almodóvar', 'España');
insert into peliculas values
(10, 'Oppenheimer', 'Drama', 1),
(11, 'Inception', 'Ciencia Ficción', 1),
(20, 'Jurassic Park', 'Aventura', 2),
(30, 'Barbie', 'Comedia', 3),
(40, 'Dolor y Gloria', 'Drama', 4),
(50, 'Película Indie', 'Indie', NULL); # Sin director famoso registrado
insert into salas values
(101, 'Sala IMAX', 10, 300),
(102, 'Sala 2', 10, 150),
(103, 'Sala VIP', 30, 50),
(104, 'Sala 4', NULL, 200), # Sala vacía (sin película)
(105, 'Sala 5', 99, 100); # Sala con ID de película que no existe

```

Realizar las siguientes sentencias:

- Muestra el nombre de la sala, el título de la película y el nombre de su director.
- `SELECT s.nombre_sala, p.titulo, d.nombre_director FROM salas s INNER JOIN peliculas p ON s.id_pelicula = p.id_pelicula INNER JOIN directores d ON p.id_director = d.id_director;`

IES Isidra de Guzmán	
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 4: Realización de consultas	Consultas multitable

-
-
- Obtén una lista única de los nombres de directores que tienen películas proyectándose en alguna sala ahora mismo.
- `SELECT DISTINCT d.nombre_director FROM salas s INNER JOIN peliculas p ON s.id_pelicula = p.id_pelicula INNER JOIN directores d ON p.id_director = d.id_director;`
- Muestra todas las salas de cine y el título de la película que proyectan. Si la sala está vacía, debe aparecer el título como "<sin película>".
- `SELECT s.nombre_sala, IFNULL(p.titulo, '<sin película>') AS 'titulo' FROM salas s LEFT JOIN peliculas p ON s.id_pelicula = p.id_pelicula;`

Calcula la capacidad total (suma de asientos) de todas las salas donde se están proyectando películas dirigidas por 'Christopher Nolan'.

`SELECT SUM(s.capacidad) AS 'Capacidad Total' FROM salas s INNER JOIN peliculas p ON s.id_pelicula = p.id_pelicula INNER JOIN directores d ON p.id_director = d.id_director WHERE d.nombre_director = 'Christopher Nolan';`

Lista los títulos de las películas y los nombres de las salas donde el director sea de 'EEUU'.

`SELECT p.titulo, s.nombre_sala FROM peliculas p INNER JOIN salas s ON p.id_pelicula = s.id_pelicula INNER JOIN directores d ON p.id_director = d.id_director WHERE d.pais_origen = 'EEUU';`

Actividad 4

Dada la siguiente tabla y datos:

```
create table clientes (
id_cliente int primary key,
nombre varchar(50),
pais_residencia varchar(30)
); create table productos (
id_producto int primary key,
```

IES Isidra de Guzmán	
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 4: Realización de consultas	Consultas multitable

```

nombre_prod varchar(50),
precio decimal(10,2),
categoria varchar(20)
); create table transportistas (      id_transp int
primary key,      empresa varchar(50),      metodo_envio
varchar(20) -- aéreo, marítimo, terrestre
); create table
pedidos (
id_pedido int primary
key,      id_cliente
int,      id_producto
int,      id_transp
int,      fecha_pedido
date,      foreign key
(id_cliente)
references
clientes(id_cliente),
foreign key
(id_producto)
references
productos(id_producto)
,      foreign key
(id_transp) references
transportistas(id_tran
sp)
);
insert into clientes values (1, 'Alice Smith', 'USA'), (2, 'Hans Müller',
'Alemania'), (3, 'Yuki Tanaka', 'Japón');
insert into productos values (10, 'Smartphone X', 800, 'Tech'), (20, 'Laptop
Pro', 1200, 'Tech'), (30, 'Cafetera', 150, 'Hogar');

```

IES Isidra de Guzmán	
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 4: Realización de consultas	Consultas multitable

•

•

```
insert into transportistas values (100, 'DHL', 'Aéreo'), (200, 'Maersk', 'Marítimo'), (300, 'FedEx', 'Aéreo');
```

```
insert into pedidos values
(501, 1, 10, 100, '2024-05-01'),
(502, 1, 20, 100, '2024-05-02'),
(503, 2, 10, 200, '2024-05-05'), (504,
3, 30, 300, '2024-05-10');
```

Realizar las siguientes sentencias:

- Muestra el id_pedido, el nombre del cliente, el nombre del producto y la empresa de transporte de todos los pedidos.
- `SELECT pe.id_pedido, c.nombre, p.nombre_prod, t.empresa FROM pedidos pe INNER JOIN clientes c ON pe.id_cliente = c.id_cliente INNER JOIN productos p ON pe.id_producto = p.id_producto INNER JOIN transportistas t ON pe.id_transp = t.id_transp;`
- Lista los nombres de los clientes que han comprado productos de la categoría 'Tech' y cuyo envío ha sido 'Aéreo'.
- `SELECT c.nombre FROM pedidos pe INNER JOIN clientes c ON pe.id_cliente = c.id_cliente INNER JOIN productos p ON pe.id_producto = p.id_producto INNER JOIN transportistas t ON pe.id_transp = t.id_transp WHERE p.categoria = 'Tech' AND t.metodo_envio = 'Aéreo';`
- Queremos saber qué productos (nombre) se han enviado a 'USA' usando el método 'Aéreo'.
- `SELECT p.nombre_prod FROM pedidos pe INNER JOIN clientes c ON pe.id_cliente = c.id_cliente INNER JOIN productos p ON pe.id_producto = p.id_producto INNER JOIN transportistas t ON pe.id_transp = t.id_transp WHERE c.pais_residencia = 'USA' AND t.metodo_envio = 'Aéreo';`

Obtén una lista sin repetición de los países de residencia de los clientes que han usado a 'Maersk' como transportista.

```
SELECT DISTINCT c.pais_residencia FROM pedidos pe INNER JOIN clientes c ON
pe.id_cliente = c.id_cliente INNER JOIN transportistas t ON pe.id_transp = t.id_transp
WHERE t.empresa = 'Maersk';
```

Muestra el nombre del cliente y el nombre de la empresa de transporte, pero solo para aquellos pedidos donde el precio del producto sea mayor a 500€.

IES Isidra de Guzmán	
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 4: Realización de consultas	Consultas multitable

SELECT c.nombre, t.empresa FROM pedidos pe INNER JOIN clientes c ON pe.id_cliente = c.id_cliente INNER JOIN productos p ON pe.id_producto = p.id_producto INNER JOIN transportistas t ON pe.id_transp = t.id_transp WHERE p.precio > 500;

Actividad 5

Dada la siguiente tabla y datos:

```
create table estudios (
id_estudio int primary key,
nombre_estudio varchar(50),
sede varchar(30)
); create table plataformas ( id_plataforma int
primary key, nombre_plat varchar(30) -- pc, ps5,
xbox, switch
); create table juegos ( id_juego int primary key,
titulo varchar(100), id_estudio int, precio_base
decimal(10,2), foreign key (id_estudio) references
estudios(id_estudio)
); create table
usuarios (
id_usuario int primary
key, nickname
varchar(50), email
varchar(50)
); create table licencias ( id_licencia varchar(20) primary key,
id_usuario int, id_juego int, id_plataforma int,
fecha_compra date, foreign key (id_usuario) references
usuarios(id_usuario), foreign key (id_juego) references
```

IES Isidra de Guzmán		
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026	
Tema 4: Realización de consultas	Consultas multitable	

```

•

•

juegos(id_juego),      foreign key (id_plataforma) references
plataformas(id_plataforma)
);

insert into estudios values (1, 'FromSoftware', 'Japón'), (2, 'Naughty Dog',
'EEUU'), (3, 'CD Projekt', 'Polonia'); insert into plataformas values (10,
'PC'), (20, 'PS5'), (30, 'Xbox Series');

insert into juegos values (100, 'Elden Ring', 1, 60.00), (101, 'The Last of
Us', 2, 70.00), (102, 'Cyberpunk 2077', 3, 50.00);

insert into usuarios values (1, 'GamerPro99', 'pro@mail.com'), (2,
'ShadowLink', 'shadow@mail.com');

insert into licencias values ('X-
111', 1, 100, 10, '2024-01-10'),
('X-222', 1, 101, 20, '2024-01-12'),
('X-333', 2, 100, 20, '2024-02-01'), ('X-444',
2, 102, 10, '2024-02-05');

```

IES Isidra de Guzmán	
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 4: Realización de consultas	Consultas multitable

Realizar las siguientes sentencias:

- Muestra el nickname del usuario, el título del juego, el nombre de la plataforma y el nombre del estudio que lo desarrolló para todas las licencias vendidas.
- `SELECT u.nickname, j.titulo, p.nombre_plat, e.nombre_estudio FROM licencias l INNER JOIN usuarios u ON l.id_usuario = u.id_usuario INNER JOIN juegos j ON l.id_juego = j.id_juego INNER JOIN plataformas p ON l.id_plataforma = p.id_plataforma INNER JOIN estudios e ON j.id_estudio = e.id_estudio;`
- Lista los nicknames de los usuarios que han comprado juegos desarrollados por estudios con sede en 'Japón'.
- `SELECT DISTINCT u.nickname FROM licencias l INNER JOIN usuarios u ON l.id_usuario = u.id_usuario INNER JOIN juegos j ON l.id_juego = j.id_juego INNER JOIN estudios e ON j.id_estudio = e.id_estudio WHERE e.sede = 'Japón';`
- Muestra los títulos de los juegos (sin repetición) que han sido comprados por usuarios para la plataforma 'PC'.
- `SELECT DISTINCT j.titulo FROM licencias l INNER JOIN juegos j ON l.id_juego = j.id_juego INNER JOIN plataformas p ON l.id_plataforma = p.id_plataforma WHERE p.nombre_plat = 'PC';`
- Encuentra el email de los usuarios que poseen una licencia de 'Elden Ring' pero específicamente en la plataforma 'PS5'.
- `SELECT u.email FROM licencias l INNER JOIN usuarios u ON l.id_usuario = u.id_usuario INNER JOIN juegos j ON l.id_juego = j.id_juego INNER JOIN plataformas p ON l.id_plataforma = p.id_plataforma WHERE j.titulo = 'Elden Ring' AND p.nombre_plat = 'PS5'`
- Muestra el nombre del estudio y el título del juego, pero solo de aquellas licencias compradas en el mes de enero'.

`SELECT e.nombre_estudio, j.titulo FROM licencias l INNER JOIN juegos j ON l.id_juego = j.id_juego INNER JOIN estudios e ON j.id_estudio = e.id_estudio WHERE MONTH(l.fecha_compra) = 1;`

`SELECT e.nombre_estudio, j.titulo FROM licencias l INNER JOIN juegos j ON l.id_juego = j.id_juego INNER JOIN estudios e ON j.id_estudio = e.id_estudio WHERE l.fecha_compra BETWEEN '2024-01-01' AND '2024-01-31';`

Actividad 6

IES Isidra de Guzmán	
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 4: Realización de consultas	Consultas multitable

Dada la siguiente tabla y datos:

```
create table cliente (
  id int unsigned
auto_increment primary key,
  nombre
varchar(100) not null,
  apellido1
varchar(100) not null,
  apellido2
varchar(100),
  ciudad varchar(100),
  categoria int unsigned
);
create table comercial (
  id int
unsigned auto_increment primary key,
  nombre
varchar(100) not null,
  apellido1
varchar(100) not null,
  apellido2 varchar(100),
  comision float
);
create table pedido (
  id int unsigned
auto_increment primary key,
  total double not null,
  fecha date,
  id_cliente int unsigned not null,
  id_comercial int unsigned not null,
foreign key
(id_cliente) references cliente(id),
foreign key
(id_comercial) references comercial(id)
);
insert into cliente values(1, 'Aarón', 'Rivero', 'Gómez', 'Almería',
100);
insert into cliente values(2, 'Adela', 'Salas', 'Díaz', 'Granada',
200);
insert into cliente values(3, 'Adolfo', 'Rubio', 'Flores', 'Sevilla',
NULL);
insert into cliente values(4, 'Adrián', 'Suárez', NULL, 'Jaén', 300);
insert into cliente values(5, 'Marcos', 'Loyola', 'Méndez', 'Almería', 200);
insert into cliente values(6, 'María', 'Santana', 'Moreno', 'Cádiz', 100);
insert into cliente values(7, 'Pilar', 'Ruiz', NULL, 'Sevilla', 300);
insert
into cliente values(8, 'Pepe', 'Ruiz', 'Santana', 'Huelva', 200);
insert into
cliente values(9, 'Guillermo', 'López', 'Gómez', 'Granada', 225);
```

IES Isidra de Guzmán		
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026	
Tema 4: Realización de consultas	Consultas multitable	

```

insert into cliente values(10, 'Daniel', 'Santana', 'Loyola', 'Sevilla',
125); insert into comercial VALUES(1, 'Daniel', 'Sáez', 'Vega', 0.15);
insert into comercial VALUES(2, 'Juan', 'Gómez', 'López', 0.13); insert
into comercial VALUES(3, 'Diego','Flores', 'Salas', 0.11); insert into
comercial VALUES(4, 'Marta','Herrera', 'Gil', 0.14);
insert into comercial VALUES(5, 'Antonio','Carretero', 'Ortega', 0.12);
insert into comercial VALUES(6, 'Manuel','Domínguez', 'Hernández', 0.13);
insert into comercial VALUES(7, 'Antonio','Vega', 'Hernández', 0.11); insert
into comercial VALUES(8, 'Alfredo','Ruiz', 'Flores', 0.05);

insert into pedido values(1, 150.5, '2017-10-05', 5, 2);
insert into pedido values(2, 270.65, '2016-09-10', 1, 5);
insert into pedido values(3, 65.26, '2017-10-05', 2, 1);
insert into pedido values(4, 110.5, '2016-08-17', 8, 3);
insert into pedido values(5, 948.5, '2017-09-10', 5, 2);
insert into pedido values(6, 2400.6, '2016-07-27', 7, 1);
insert into pedido values(7, 5760, '2015-09-10', 2, 1);
insert into pedido values(8, 1983.43, '2017-10-10', 4, 6);
insert into pedido values(9, 2480.4, '2016-10-10', 8, 3);
insert into pedido values(10, 250.45, '2015-06-27', 8, 2);
insert into pedido values(11, 75.29, '2016-08-17', 3, 7);
insert into pedido values(12, 3045.6, '2017-04-25', 2, 1);
insert into pedido values(13, 545.75, '2019-01-25', 6, 1);
insert into pedido values(14, 145.82, '2017-02-02', 6, 1);
insert into pedido values(15, 370.85, '2019-03-11', 1, 5);
insert into pedido values(16, 2389.23, '2019-03-11', 1, 5);

```

Realizar las siguientes sentencias:

- Devuelve un listado de todos los clientes que realizaron un pedido durante el año 2017, cuya cantidad esté entre 300 € y 1000 €.

IES Isidra de Guzmán	
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 4: Realización de consultas	Consultas multitable

- `SELECT DISTINCT c.nombre, c.apellido1, c.apellido2 FROM cliente c INNER JOIN pedido p ON c.id = p.id_cliente WHERE YEAR(p.fecha) = 2017 AND p.total BETWEEN 300 AND 1000;`
- Devuelve un listado con todos los comerciales junto con los datos de los pedidos que han realizado. Este listado también debe incluir los comerciales que no han realizado ningún pedido. El listado debe estar ordenado alfabéticamente por el primer apellido, segundo apellido y nombre de los comerciales.
- `SELECT co.nombre, co.apellido1, co.apellido2, p.id AS id_pedido, p.total, p.fecha FROM comercial co LEFT JOIN pedido p ON co.id = p.id_comercial ORDER BY co.apellido1 ASC, co.apellido2 ASC, co.nombre ASC;`
- Devuelve un listado que solamente muestre los clientes que no han realizado ningún pedido.
- `SELECT c.nombre, c.apellido1, c.apellido2 FROM cliente c LEFT JOIN pedido p ON c.id = p.id_cliente WHERE p.id IS NULL;`
- Devuelve un listado con los clientes que no han realizado ningún pedido y de los comerciales que no han participado en ningún pedido. Ordene el listado alfabéticamente por los apellidos y el nombre. En el listado deberá diferenciar de algún modo los clientes y los comerciales.
- `SELECT c.nombre, c.apellido1, c.apellido2, 'Cliente' AS Tipo FROM cliente c LEFT JOIN pedido p ON c.id = p.id_cliente WHERE p.id IS NULL UNION SELECT co.nombre, co.apellido1, co.apellido2, 'Comercial' AS Tipo FROM comercial co LEFT JOIN pedido p ON co.id = p.id_comercial WHERE p.id IS NULL ORDER BY apellido1 ASC, apellido2 ASC, nombre ASC;`
- Calcula cuál es el máximo valor de los pedidos realizados durante el mismo día para cada uno de los clientes, teniendo en cuenta que sólo queremos mostrar aquellos pedidos que superen la cantidad de 2000 €.
- `SELECT c.nombre, c.apellido1, c.apellido2, p.fecha, MAX(p.total) AS Maximo_Valor FROM cliente c INNER JOIN pedido p ON c.id = p.id_cliente GROUP BY c.id, p.fecha HAVING MAX(p.total) > 2000;`
- Devuelve un listado con el identificador de cliente, nombre y apellidos y el número total de pedidos que ha realizado cada uno de los clientes. Tenga en cuenta que pueden existir clientes que no han realizado ningún pedido. Estos clientes también deben aparecer en el listado indicando que el número de pedidos realizados es 0.
- `SELECT c.id, c.nombre, c.apellido1, c.apellido2, COUNT(p.id) AS Total_Pedidos FROM cliente c LEFT JOIN pedido p ON c.id = p.id_cliente GROUP BY c.id;`
- Devuelve cuál ha sido el pedido de máximo valor que se ha realizado cada año.
- `SELECT YEAR(fecha) AS 'Año', MAX(total) AS 'Maximo_Valor' FROM pedido GROUP BY YEAR(fecha);`

IES Isidra de Guzmán		
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026	
Tema 4: Realización de consultas	Consultas multitable	

5. Otros operadores

5.1. intersect

El operador `intersect` devuelve las filas comunes entre dos consultas, por defecto elimina duplicados (se pueden tener duplicados si se especifica `all`), su sintaxis es:

```
select ... intersect
[all] select ...
```

Por ejemplo:

```
create table t1 (
id int primary key
);

create table t2 like
t1;

insert into t1(id)
values(1),(2),(3);

insert into t2(id)
values(2),(3),(4);

select id from t1
intersect select
id from t2;
```

Devolverá:

```
+-----+
| id |
+-----+
```

IES Isidra de Guzmán	
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 4: Realización de consultas	Consultas multitable

```
| 2 |
| 3 |
+----+
```

5.2. except (minus)

El operador `except` (también se le denomina `minus` en otros SBDB) compara el resultado de dos consultas y devuelve las filas de la primera consulta que no se encuentran en la segunda. Su sintaxis es:

```
select ... minus
select ...
```

Por ejemplo:

```
select id from t1
except select id
from t2;
```

Devolverá:

```
+----+
| id |
+----+
| 1 |
+----+
```

IES Isidra de Guzmán		
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026	
Tema 4: Realización de consultas	Consultas multitable	

5.3. union

El operador `union` devuelve el resultado de 2 consultas, es decir, las filas de la primera consulta y las de la segunda consulta. Por defecto elimina duplicados (se pueden tener duplicados si se especifica `all`), su sintaxis es:

```
select ... union
[all] select
...
```

Por ejemplo:

```
select id from t1
union select id
from t2;
```

Devolverá:

```
+----+
| id |
+----+
| 1 |
| 2 |
| 3 |
| 4 |
+----+
```

IES Isidra de Guzmán	
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 4: Realización de consultas	Consultas multitable

Actividad 7

Dada la siguiente tabla y datos:

```

create table estudiantes (
id_estudiante int primary key,
nombre varchar(50) not null,
carrera varchar(50) not null
); create table
asignaturas (
id_asignatura int primary
key,      nombre
varchar(50) not null,
semestre int not null
); create table matriculas (      id_estudiante int,      id_asignatura
int,      nota decimal(4,2),      primary key (id_estudiante,
id_asignatura),      foreign key (id_estudiante) references
estudiantes(id_estudiante),      foreign key (id_asignatura) references
asignaturas(id_asignatura)
);

insert into estudiantes values (1, 'Ana Torres', 'Ingenieria');
insert into estudiantes values (2, 'Luis Perez', 'Ingenieria');
insert into estudiantes values (3, 'Maria Lopez', 'Administración');
insert into estudiantes values (4, 'Carlos Ruiz', 'Administración');
insert into estudiantes values (5, 'Elena Gomez', 'Ingenieria');
insert into estudiantes values (6, 'Jorge Diaz', 'Derecho'); -- no
matriculado

insert into asignaturas values (101, 'Matematicas', 1); insert into
asignaturas values (102, 'Programacion', 1); insert into asignaturas
values (103, 'Contabilidad', 1); insert into asignaturas values (201,

```

IES Isidra de Guzmán	
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 4: Realización de consultas	Consultas multitable

```
'Base de Datos', 2); insert into asignaturas values (202, 'Estadística',
2); insert into asignaturas values (203, 'Marketing', 2); insert into
asignaturas values (300, 'Filosofía', 3); -- sin estudiantes insert
into matriculas values (1, 101, 8.5); insert into matriculas values (1,
201, 7.0);

insert into matriculas values (2, 101,
4.0); insert into matriculas values (2, 102,
6.5);

insert into matriculas values (3, 103,
9.0); insert into matriculas values (3, 203,
8.0);

insert into matriculas values (4, 103,
3.5); insert into matriculas values (4, 202,
5.5);

insert into matriculas values (5, 102,
7.5); insert into matriculas values (5, 202,
9.0);
```

Realizar las siguientes sentencias usando UNION, INTERSECT y EXCEPT:

- Los nombres de estudiantes de "ingeniería" o de "Administración".
- Se usa UNION para juntar los resultados de dos select distintas.
- SELECT nombre FROM estudiantes WHERE carrera = 'Ingeniería' UNION SELECT nombre FROM estudiantes WHERE carrera = 'Administración';
- Estudiantes que estén matriculados tanto en asignaturas del primer como de segundo trimestre.
- Usamos INTERSECT: Sacamos los de semestre 1 y nos quedamos solo con los que también están en el 2.
- SELECT e.nombre FROM estudiantes e INNER JOIN matriculas m ON e.id_estudiante = m.id_estudiante INNER JOIN asignaturas a ON m.id_asignatura = a.id_asignatura WHERE a.semestre = 1 INTERSECT SELECT e.nombre FROM estudiantes e INNER JOIN matriculas m ON

IES Isidra de Guzmán	
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 4: Realización de consultas	Consultas multitable

e.id_estudiante = m.id_estudiante INNER JOIN asignaturas a ON m.id_asignatura = a.id_asignatura WHERE a.semestre = 2;

- Estudiantes que tengan alguna nota mayor a 8 y sean de ingeniería.
- SELECT e.nombre FROM estudiantes e INNER JOIN matriculas m ON e.id_estudiante = m.id_estudiante WHERE m.nota > 8 INTERSECT SELECT nombre FROM estudiantes WHERE carrera = 'Ingenieria';
- Estudiantes matriculados que ya tengan todas las asignaturas aprobadas.
- Lógica: Todos los matriculados MENOS (EXCEPT) los que tienen alguna nota menor de 5.
- SELECT e.nombre FROM estudiantes e INNER JOIN matriculas m ON e.id_estudiante = m.id_estudiante EXCEPT SELECT e.nombre FROM estudiantes e INNER JOIN matriculas m ON e.id_estudiante = m.id_estudiante WHERE m.nota < 5;
- Asignaturas que no tengan estudiantes matriculados.
- Lógica: Todas las asignaturas MENOS (EXCEPT) las que aparecen en la tabla matrículas.

SELECT nombre FROM asignaturas EXCEPT SELECT a.nombre FROM asignaturas a INNER JOIN matriculas m ON a.id_asignatura = m.id_asignatura;

INTERSECT es tu "Y" lógico entre bloques masivos: En lugar de hacer subconsultas o JOINS complejos, el apartado 3.2 calcula dos listas separadas (los del semestre 1 por un lado, los del semestre 2 por otro) y el INTERSECT actúa como un colador, dejando pasar solo los nombres que están repetidos en ambas listas

EXCEPT es tu botón de "eliminar": El apartado 3.4 es la forma más elegante de resolver el "que tengan todo aprobado". Podrías volverte loco agrupando y contando notas medias, pero es mucho más rápido decirle a SQL: "Dame la lista de todos los que están apuntados a algo, y ahora bórrame de esa lista a cualquiera que haya suspendido algo". Los que sobreviven a la criba, son los que lo tienen todo aprobado